

Implementación de modelo de redes neuronales convolucionales para clasificación de imágenes hiperespectrales usando transformada wavelet

Brayan Yesid Quintero Santander
Ingeniería de Sistemas, Universidad Industrial de Santander, Colombia,
brayan2221707@correo.uis.edu.co

Cristhian Camilo Rey Rueda
Ingeniería de Sistemas, Universidad Industrial de Santander, Colombia,
cristhian2200190@correo.uis.edu.co

Juan David Toloza Parada
Ingeniería de Sistemas, Universidad Industrial de Santander, Colombia,
juan2221067@correo.uis.edu.co

Resumen

Introducción: La clasificación de imágenes hiperespectrales (HSI) mediante redes neuronales convolucionales (CNN) es un área de gran interés en la visión por computadora. Sin embargo, muchos métodos utilizados para esta tarea no integran eficazmente las características espectrales y espaciales, lo que afecta el rendimiento. En este trabajo, se presenta SpectralNET [1], una CNN que utiliza transformadas wavelet para extraer de manera eficiente dichas características, mejorando la precisión en comparación con enfoques tradicionales como SVMs, 2D CNNs y 3D CNNs. **Objetivo:** Desarrollar un modelo CNN que clasifique imágenes HSI de manera precisa y eficiente. **Metodología:** Se utilizó el dataset de la Universidad de Pavia, generando parches de 12x12x3 mediante análisis factorial y aumento de datos. Estos parches fueron utilizados como entrada en SpectralNET, el cual fue entrenado y evaluado. **Resultados:** SpectralNET mostró un excelente rendimiento en las fases de entrenamiento y validación, superando a varios métodos tradicionales. Su capacidad para extraer tanto características espectrales como espaciales le permite destacar en la clasificación de imágenes HSI. **Conclusiones:** SpectralNET mejora la precisión de clasificación y reduce la carga computacional en comparación con otros métodos. La combinación de transformadas wavelet y 2D CNN demuestra ser una solución eficaz y de alto rendimiento para la clasificación de datos hiperespectrales.

Palabras clave: imágenes hiperespectrales, clasificación espectral, redes neuronales convolucionales, transformada wavelet.

1. Introducción

Las imágenes hiperespectrales (HSI, por sus siglas en inglés) capturan información de una porción del espacio en múltiples bandas del espectro electromagnético, a diferencia de las imágenes RGB convencionales que solo utilizan tres bandas. Esto permite revelar características de los materiales que no son perceptibles al ojo humano. Del fenómeno descrito anteriormente surge el concepto de "firma espectral", única para cada material, y que se obtiene al analizar cómo dicho material refleja o absorbe la luz (reflectancia) [2]. Este artículo presenta el proceso de implementación de un modelo de redes neuronales convolucionales (CNN) que utiliza transformadas wavelet para extraer de manera eficiente la información contenida en un cubo espectral. La aplicación de estas técnicas avanzadas optimizó el procesamiento de las imágenes, permitiendo al modelo identificar con alta precisión diversos objetos o superficies, como casas y cuerpos de agua, y de esta manera realizar una clasificación detallada de las características presentes en la imagen.

2. Metodología

2.1 Descripción del modelo

Las redes neuronales convolucionales (CNN) 2D se emplean comúnmente para procesar

imágenes, ya que están diseñadas para trabajar sobre datos en dos dimensiones. Se denominan "bidimensionales" porque su núcleo o *kernel* se desplaza a lo largo de las dos dimensiones espaciales de la imagen, extrayendo de manera eficiente las características espaciales clave [3]. Esta capacidad de capturar patrones y relaciones espaciales en los datos es una de las principales ventajas de las CNNs, superando a otros tipos de redes que no están optimizadas para este tipo de tareas [4]. Una CNN básica puede representarse matemáticamente como la suma ponderada de vecinos más próximos con un sesgo constante añadido.

Dado un vector de entrada X_n con las etiquetas correspondientes Y_n de el espacio R^n . En la Ecuación 1, Y_i es una etiqueta de las etiquetas Y_n y X_i es la muestra correspondiente de las muestras X_n . W_j es el peso definido por el núcleo. N_i son los datos i vecinos.

$$y_i = \sum_{j \in N_i} W_j X_j \quad (1)$$

La ecuación 1 puede ser simplemente considerada como la convolución de X_i y el *kernel* W_j puede reescribirse como

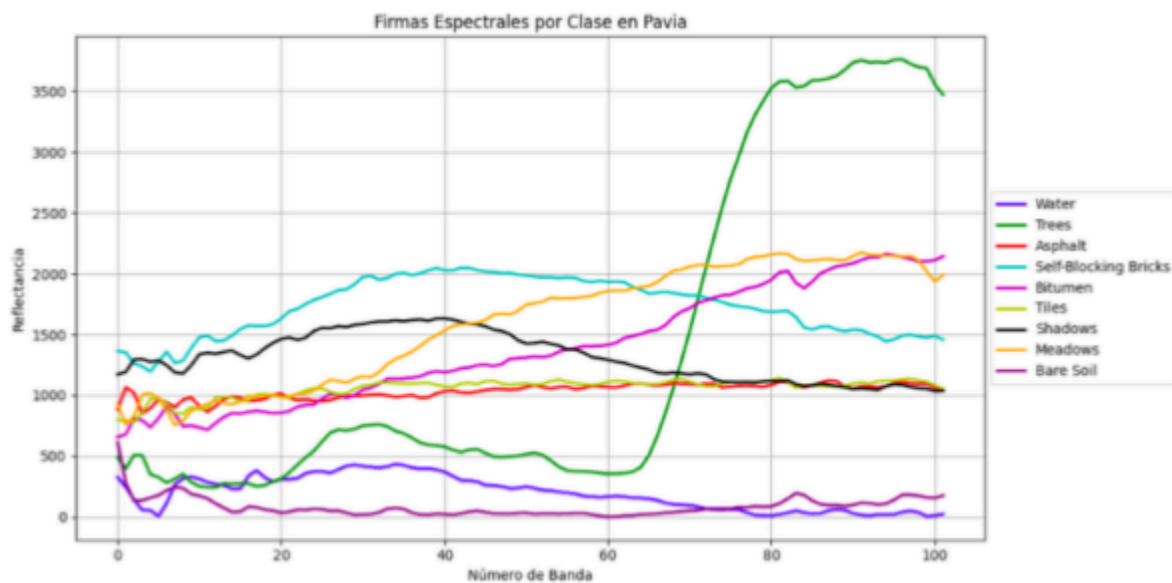


Figura 1: Firmas espectrales correspondientes a las 9 clases etiquetadas en el dataset. En la imagen, se muestran las firmas espectrales asociadas a cada clase, lo que permite visualizar las diferencias y patrones característicos entre las distintas categorías del conjunto de datos.

$Y = X * W$, esto se llama capa de convolución donde W se encuentra en R^0 . La salida de las capas de convolución suele ser grande y necesita ser agrupado antes de alimentarlo en las siguientes capas. Las capas de agrupación (del inglés *Pooling Layer*), se colocan entre capas de convolución para realizar una operación de filtrado y reducir el número de salidas.

2.2 Preprocesamiento de los datos

El cubo hiperspectral que utilizamos como entrada a nuestro modelo tenía inicialmente dimensiones de $1096 \times 715 \times 102$. En la Figura 1, observamos las firmas espectrales de cada una de las clases de nuestro conjunto de datos contaba con 9 clases etiquetadas Usando una técnica estadística llamada análisis de factorial, logramos reducir la dimensionalidad espectral de nuestro cubo a $1096 \times 715 \times 3$. Las dimensiones espectrales se preservan en el análisis

factorial, es decir, $M \times N$, pero las bandas se reducen de 102 a 3. Utilizar el análisis factorial en el cubo hiperspectral como paso de preprocesamiento es extremadamente beneficioso, ya que esta técnica es capaz de describir la variabilidad entre las diferentes bandas correlacionadas y solapadas, lo que ayuda a que el modelo clasifique mejor ejemplos similares. Tras completar el paso de análisis factorial, se extrajeron parches 3D superpuestos de dimensiones $12 \times 12 \times 3$ del cubo hiperspectral preprocesado. El tamaño de los parches de 12×12 se determinó mediante experimentación con el fin de optimizar la precisión global del modelo. Posteriormente, los parches fueron sometidos a un proceso de aumento de datos, en el que se aplicaron rotaciones, incrementando el número de muestras de entrada en un 40%. El conjunto de datos resultante, tras el análisis factorial y el aumento de datos, fue

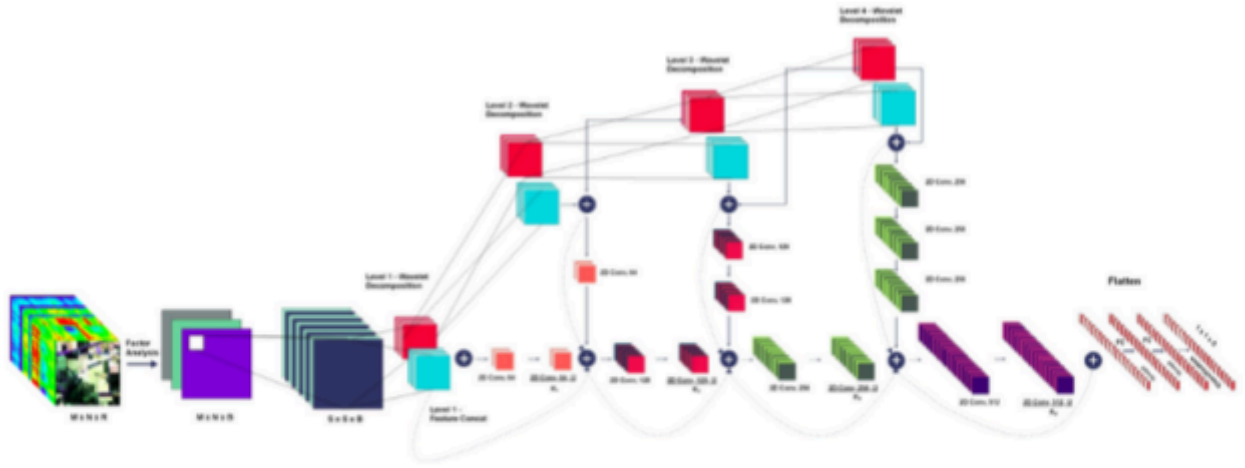


Figura 2: El cubo HSI de entrada se preprocesa mediante Análisis Factorial (FA) para reducir su dimensión a 3 componentes. Posteriormente, se extraen parches de la imagen y se someten a un proceso de aumento de datos, que luego se utilizan como entrada al modelo SpectralNET. Este modelo aplica una descomposición *wavelet* en 4 niveles. Utiliza núcleos de 3x3 con *padding* de 1x1 y *stride* de 2 para reducir el mapa de características, agregando las características transformadas por canal. Para mitigar la desaparición del gradiente, se emplean atajos mediante convoluciones 1x1. Finalmente, se aplica un *pooling* global antes de enviar la salida a las capas completamente conectadas con *dropout*.

utilizado como entrada en nuestro modelo *SpectralNET*. En la Figura 2 podemos observar detalladamente la arquitectura del modelo implementado.

2.3 Fundamentos de la transformada *wavelet*

Esto abre el camino hacia las CNN de múltiples resoluciones, donde la convolución es realizada por un par de *kernels* K_{low} y K_{high} , los cuales generan X_{low} y X_{high} . La CNN de múltiples resoluciones realiza una descomposición jerárquica de $X_{low,t}$ en $X_{low,t+1}$ y $X_{high,t+1}$ con *kernels* diferentes en cada paso t [5]. En *SpectralNet*, el *kernel* de onda $K_{high,t}$ se basa en las ondas Haar (del inglés *Haar wavelet*), mientras que $K_{low,t}$ corresponde a una función de escalado. Las ondas Haar en 2D utilizan cuatro *kernels*

específicos: $f_{L,L}$, $f_{L,H}$, $f_{H,L}$, $f_{H,H}$, que se emplean para llevar a cabo la transformada *wavelet* [6].

$$\begin{aligned} f_{L,L} &= \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} & f_{L,H} &= \begin{bmatrix} -1 & -1 \\ 1 & 1 \end{bmatrix} \\ f_{H,L} &= \begin{bmatrix} -1 & 1 \\ -1 & 1 \end{bmatrix} & f_{H,H} &= \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \end{aligned} \quad (2)$$

En un parche de la imagen hiperespectral HSI, al aplicarle la transformada de Haar, el valor de la posición espectral (i,j) puede expresarse como:

$$\begin{aligned} Haar(i,j) &= x(2i-1, 2j-1) + x(2i-1, 2j) \\ &\quad + x(2i, 2j-1) + x(2i, 2j) \end{aligned} \quad (3)$$

El parche del HSI es tomado como entrada y descompuesta por la transformada *wavelet* en sub-bandas, después estas sub-bandas son enviadas a una capa de convolución para que el modelo aprenda de las características espectrales y su localización. Posteriormente, una parte de la sub-banda es descompuesta nuevamente en la siguiente capa mediante la

transformada *wavelet*, que posteriormente se envía a la capa de convolución. Este proceso de descomposición se repite en cada capa, lo que permite que la red neuronal convolucional aprenda progresivamente las características espectrales y espaciales del parche hiperspectral. De esta manera, la CNN aprovecha tanto la información espacial como la espectral para una mejor representación y clasificación de los datos.

3. Resultados

3.1 Dataset y Entrenamiento

El experimento fue realizado utilizando el dataset de la Universidad de Pavia, un conjunto de datos ampliamente utilizado en el procesamiento de imágenes hiperspectrales. Este dataset consta de un cubo espectral con 1,096 filas, 715 columnas y 102 bandas espectrales, cubriendo un rango de longitud de onda de 430 a 860 nm, lo que lo convierte en una fuente de datos rica y variada para la clasificación espectral.

El entorno de trabajo seleccionado para llevar a cabo el entrenamiento fue Google Colab, aprovechando su ejecución gratuita en la GPU T4, que permitió reducir significativamente el tiempo de procesamiento y entrenamiento del modelo. Esta elección fue clave para acelerar los experimentos, dado que las tareas de clasificación en imágenes hiperspectrales pueden ser computacionalmente intensivas.

Basándonos en estudios previos y en la literatura científica relevante, se decidió configurar el optimizador de descenso de gradiente estocástico (SGD, por sus siglas en inglés) con una tasa de aprendizaje de 0.01 y un momentum de 0.9. Estas configuraciones fueron seleccionadas porque ofrecieron un buen equilibrio entre la velocidad de convergencia y la estabilidad durante el entrenamiento, evitando tanto la convergencia prematura como la oscilación excesiva del gradiente. La combinación de estos parámetros se ha demostrado eficaz para optimizar la minimización del error durante el proceso de ajuste de los pesos del modelo.

Para mejorar la eficiencia del entrenamiento y reducir el tiempo de procesamiento, se optó por extraer parches con dimensiones de 12x12 píxeles y un número reducido de 3 bandas espectrales por parche. Esta reducción en la cantidad de bandas fue motivada por la necesidad de disminuir la complejidad computacional, sin comprometer significativamente la información espectral útil para la clasificación. Esta estrategia ha sido empleada en varios estudios para encontrar un equilibrio entre la dimensionalidad de los datos y el rendimiento del modelo. El modelo fue entrenado durante un total de 10 épocas, un número que se estimó suficiente para lograr una convergencia del modelo sin llegar a un sobreajuste. Cada época representó un ciclo completo de entrenamiento donde el

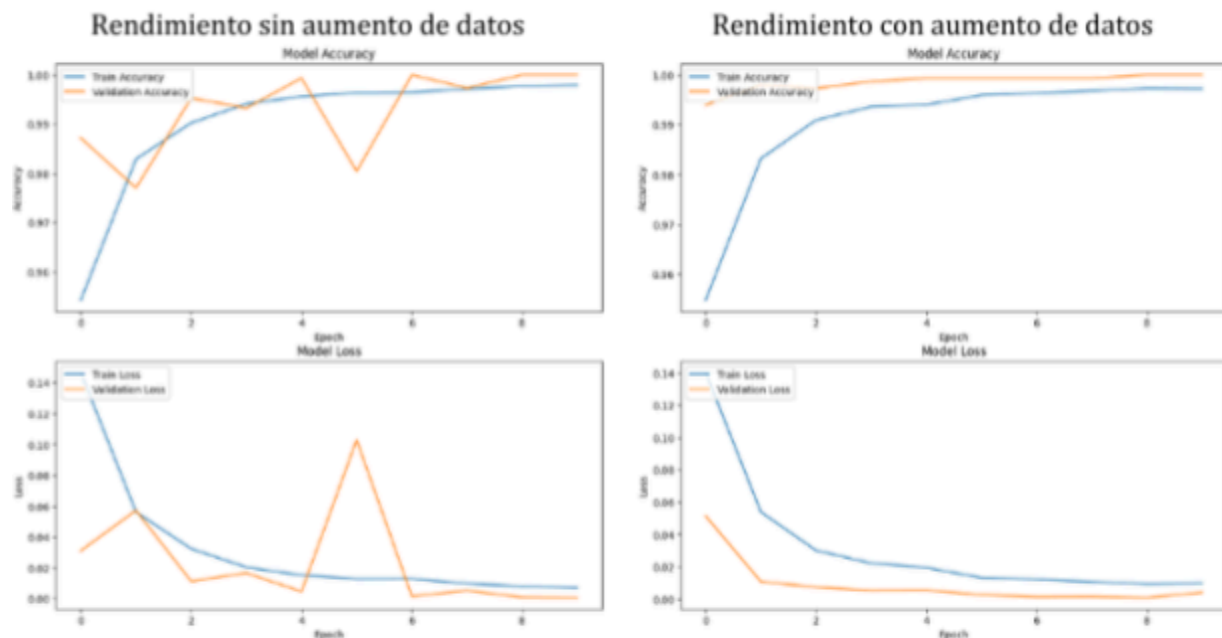


Figura 3: Gráficas de rendimiento del modelo. En la parte izquierda, se presentan las gráficas que ilustran el comportamiento del modelo en términos de precisión (*accuracy*) y pérdida (*loss*) utilizando parches de entrada de dimensiones 12x12x3. En la parte derecha, se muestra el rendimiento del modelo después de aplicar un proceso de aumento de datos (*data augmentation*) sobre dichos parches. Este enfoque permitió optimizar tanto la precisión como la reducción de la pérdida, mejorando significativamente el rendimiento general del modelo en ambas métricas.

modelo ajustó sus parámetros para minimizar la función de pérdida en el conjunto de entrenamiento.

3.2 Resultados de clasificación

El rendimiento del modelo se presenta en la Figura 3, donde se muestran las gráficas del comportamiento del *accuracy* y la *loss* a lo largo de las diferentes épocas. Estas métricas ofrecen una evaluación precisa del desempeño del modelo tanto durante la fase de entrenamiento como en la etapa de prueba con los datos de validación. La partición de los datos se realizó dividiendo el conjunto en un 80% para entrenamiento y un 20% para prueba, garantizando así un adecuado balance entre la cantidad de datos usados para ajustar los parámetros del modelo y un conjunto suficientemente representativo para

evaluar su capacidad de generalización. Tal como se observa en la Figura 3, el modelo alcanzó un rendimiento notable, logrando altos niveles de *accuracy* tanto en el entrenamiento como en la validación. Las curvas de *accuracy* y *loss* revelan una mejora progresiva a lo largo de las épocas, lo que sugiere que el modelo está convergiendo de manera efectiva hacia una solución óptima. Además, la implementación de técnicas de aumento de datos (*data augmentation*) contribuyó significativamente a mejorar el rendimiento del modelo, reflejando una mejor generalización y una reducción del riesgo de sobreajuste (*overfitting*).

La mejora en el rendimiento es evidente en la mayor estabilidad observada en las gráficas, donde se aprecia una disminución más rápida de la *loss* y un aumento más consistente del

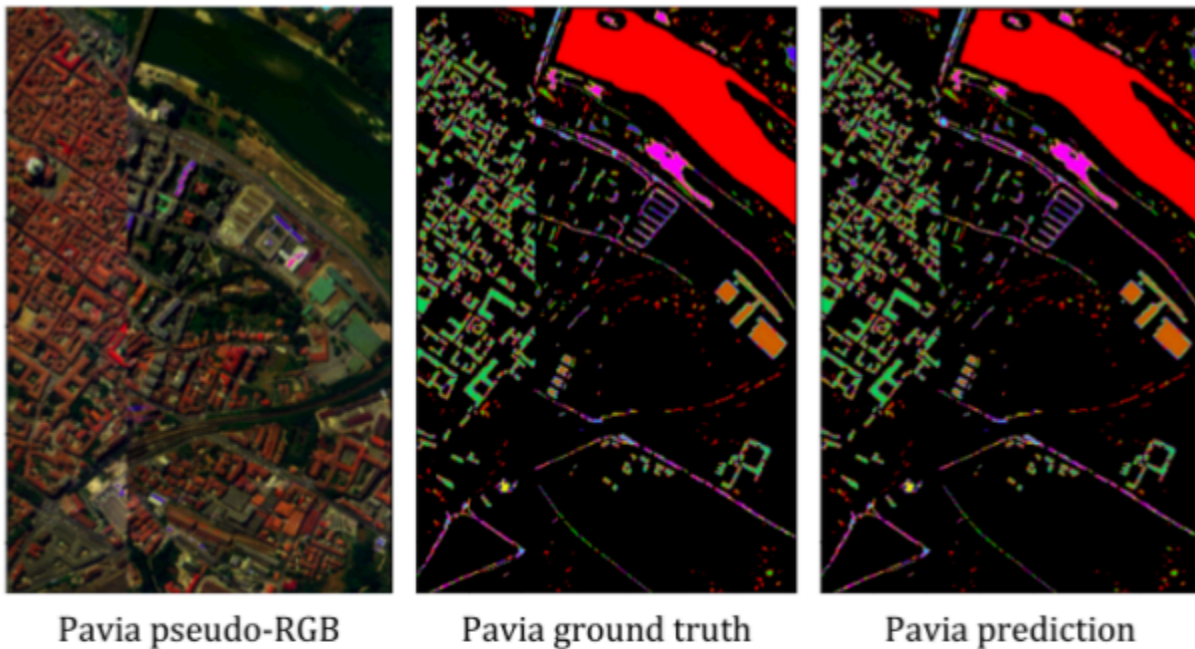


Figura 4: Comparación de imágenes del dataset Pavia. A la izquierda, se muestra una imagen pseudo-RGB de la ciudad de Pavia, generada a partir de un conjunto de datos hiperespectrales. En el centro, se presenta el *ground truth*, donde se muestran las clases de nuestro cubo hiperespectral clasificadas correctamente. A la derecha, se observa la predicción realizada por el modelo implementado, que reproduce con alta precisión las diferentes clases presentes en el *ground truth*, evidenciando la efectividad del modelo en la clasificación de los datos hiperespectrales.

accuracy a lo largo de las épocas. Esto indica que el modelo no solo logra adaptarse correctamente a los datos de entrenamiento, sino que también mantiene un desempeño robusto y confiable cuando se enfrenta a datos no vistos previamente. Estas observaciones resaltan la importancia de técnicas como el data augmentation para promover un entrenamiento más eficaz y equilibrado, evitando que el modelo se ajuste demasiado a los datos específicos del conjunto de entrenamiento y favoreciendo una generalización más sólida en diferentes escenarios de testeo. En la Figura 4 podemos observar resultados visuales de la predicción y su respectiva comparación con el *ground truth*.

4. Discusión

Además de la implementación del modelo de SpectralNet se logró un manejo apropiado

para los datos que permitió modificar y mejorar el preprocesamiento, lo que ayudó a reducir el riesgo de sobreajuste, especialmente considerando la cantidad limitada de parámetros en el conjunto de datos utilizado. También se aplicaron predicciones por lotes (del inglés *batch predictions*), para optimizar el rendimiento y aligerar la carga computacional durante el proceso de predicción. Una de nuestras principales limitaciones fue el tamaño del conjunto de datos, que acota el análisis a un número limitado de parámetros. Durante la implementación nos enfrentamos a un desafío relacionado con el tamaño de los parches. Optamos por el método de prueba y error para determinar el tamaño óptimo, ya que:

- Cuando seleccionamos parches más pequeños, experimentamos

problemas de memoria en Google colab notebooks, impidiendo así continuar con el entrenamiento del modelo.

- En caso de seleccionar un tamaño de parche mayor al utilizado, el modelo se veía limitado debido a la reducción en la cantidad de datos de entrada, lo que aumentaba el riesgo de sobreajuste.

5. Conclusiones

En este artículo, se presentó una implementación efectiva de la arquitectura *SpectralNet* para la clasificación de imágenes hiperespectrales. Los resultados demuestran que la combinación de estas técnicas de preprocesamiento mejoran significativamente el rendimiento del modelo, logrando adaptarse eficientemente al conjunto de datos hiperespectrales. Las métricas obtenidas, particularmente en términos de precisión y capacidad predictiva, son notablemente altas y corresponden fielmente a las etiquetas proporcionadas en el dataset. Estos hallazgos sugieren que *SpectralNet* es una arquitectura prometedora para tareas de clasificación de imágenes hiperespectrales. Futuras líneas de investigación podrían explorar la aplicación de este enfoque a otros conjuntos de datos y tareas de aprendizaje automático.

Referencias

[1] Chakraborty, T., & Trehan, U. (2021).

SpectralNET: Exploring Spatial-Spectral WaveletCNN for Hyperspectral Image Classification. *arXiv*, 2104(00341v1), 1-7. Recuperado en 10 de septiembre de 2024, de <https://arxiv.org/abs/2104.00341>

[2] Amigo, J. M. (2020). Hyperspectral and multispectral imaging: setting the scene. En J. M. Amigo (Ed.), *Hyperspectral Imaging* (pp. 3-16). Países Bajos: Elsevier

[3] J. Zheng, Y. Feng, C. Bai, and J. Zhang. (2020). Hyperspectral image classification using mixed convolutions and covariance pooling. *IEEE Transactions on Geoscience and Remote Sensing* (pp. 1-13).

[4] Ge, Z., Cao, G., Li, X., & Fu, P. (2020). Hyperspectral image classification method based on 2D–3D CNN and multibranch feature fusion. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 13, 5776-5788.

[5] Cheng, M.-M., Jiang, P.-T., Han, L.-H., Wang, L., & Torr, P. (2023). Deeply explain CNN via hierarchical decomposition. *International Journal of Computer Vision*. Recuperado de https://mftp.mmcheng.net/Papers/23IJCV_CN_NasDT.pdf

[6] Fujieda, S., Takayama, K., & Hachisuka, T. (2018). Wavelet convolutional neural networks. *arXiv preprint*, arXiv:1805.08620.